Title of Invention: System and Method for In-Memory Infrastructure Resilience Simulation Using Graph-Based Cascade Propagation with Multi-Layer Availability Constraints and AI Agent Cross-Layer Failure Modeling

Inventor: Yutaro Maeda

Date: March 2026

Application Type: US Provisional Patent Application

Classification: G06F 11/36 (Testing software for fault injection); G06F 11/07 (Responding to faults); G06N 20/00 (Machine learning)

ABSTRACT

A computer-implemented system and method for evaluating infrastructure resilience entirely in computer memory without affecting real infrastructure. An infrastructure topology · including both traditional infrastructure components and AI agent components · is modeled as a directed graph in which nodes represent components with typed attributes and edges represent typed dependencies classified as required, optional, or asynchronous. Fault scenarios are automatically generated and failure propagation is simulated through the graph using a cascade engine whose semantics are formally defined as a Labeled Transition System (LTS) with a state tuple of (health map, latency map, elapsed time, visited set) and typed transition rules that differentiate propagation behavior based on dependency type. Per-layer availability ceilings are computed across at least five independent constraint layers · hardware, software, theoretical, operational, and external SLA · and the binding constraint layer producing the minimum availability ceiling is identified. Simulation results exceeding any layer ceiling are flagged as physically unrealizable, and infrastructure modification specifications are generated to relax the binding constraint. The system further models AI agent failure modes including hallucination probability as a computable function of infrastructure state, cross-layer cascade from infrastructure failure through data source unavailability to agent behavior degradation, and agent-to-agent cascade propagation with compound probability computation, generating infrastructure monitoring thresholds that maintain agent hallucination probability below configured acceptable levels.

---

1. FIELD OF THE INVENTION

The present invention relates to systems and methods for evaluating the resilience of computing infrastructure. More specifically, the invention pertains to an in-memory simulation system that models infrastructure topologies as directed graphs, injects virtual faults into the model without affecting any real systems, simulates failure propagation using formally-specified Labeled Transition System semantics with dependency-type-aware cascade rules, computes availability limits through a multi-layer mathematical framework that constrains simulation results by physically realizable ceilings, and simulates AI agent failure modes · including hallucination, context overflow, and agent-to-agent cascade propagation · as functions of infrastructure state, revealing emergent failure modes invisible to infrastructure monitoring and AI evaluation benchmarks operating independently.

2. BACKGROUND OF THE INVENTION

2.1 Problem Statement

Modern distributed computing systems are increasingly complex, comprising dozens to hundreds of interconnected components (load balancers, application servers, databases, caches, message queues, and external APIs). Understanding how these systems behave under failure conditions is critical to ensuring reliability.

The emergence of AI agent architectures · where LLM-powered agents orchestrate tools, consume data sources, and chain outputs to downstream agents · introduces entirely new failure modes invisible to traditional infrastructure

monitoring. When an infrastructure component (e.g., a database) fails, it may sever an AI agent's grounding data source, causing the agent to hallucinate rather than fail cleanly. This hallucination propagates silently to downstream agents and systems, compounding errors without triggering traditional alerting.

2.2 Limitations of Existing Approaches

Existing approaches to infrastructure resilience evaluation fall into several categories, all with significant limitations:

A) Real-Environment Fault Injection (Chaos Engineering)

Tools such as Netflix Chaos Monkey (2011), Gremlin, LitmusChaos, Chaos Mesh, and AWS Fault Injection Simulator inject faults into live or staging environments. These approaches require access to actual infrastructure resources, carry risk of unintended production impact, are expensive to operate, cannot evaluate theoretical availability limits, cannot exhaustively test all failure combinations, and do not model AI agent behavior.

B) Static Analysis Tools

Tools such as HPE's SPOF analysis (US9280409B2) perform static analysis of infrastructure configurations. These approaches do not simulate dynamic system behavior, cannot model time-varying conditions, do not account for the interplay between multiple failure modes, and cannot quantify the severity and propagation of cascading failures.

C) AI/ML Testing Frameworks

LLM evaluation benchmarks and red-teaming tools evaluate model outputs for hallucination, toxicity, and correctness at the prompt/response level. They do not simulate the infrastructure conditions that cause hallucinations and do not model how infrastructure failures propagate through data availability layers to affect agent behavior.

D) Graph-Based Reliability Analysis

Existing graph-based approaches to infrastructure reliability, such as Krasnovsky & Zorkin's connectivity-based model, use graph connectivity metrics (vertex connectivity, edge connectivity) to assess resilience. These approaches treat all dependencies as equivalent, do not distinguish between required, optional, and asynchronous dependency types, do not model cascade propagation dynamics using formal transition semantics, lack circuit breaker containment modeling, and cannot simulate the differential behavior of failure propagation across dependency types. They compute static graph-theoretic metrics rather than simulating dynamic failure cascade behavior.

E) Traditional Reliability Engineering Tools

Tools such as Isograph Reliability Workbench and similar RAMS (Reliability, Availability, Maintainability, Safety) software compute availability using single-layer models · typically hardware reliability (MTBF/MTTR) with redundancy configurations. They do not compute independent availability ceilings across multiple constraint layers (software, operational, theoretical noise floor, external SLA), cannot identify the binding constraint layer limiting system availability, and do not generate infrastructure modification specifications targeting the specific constraint layer that limits availability.

F) Graph Analysis for Fault Targeting

Dell's US11356324B2 performs graph analysis of infrastructure topologies to select optimal targets for fault injection into real environments. This approach uses graph metrics for target selection rather than performing in-memory failure simulation, requires access to actual infrastructure for fault execution, and does not model cascade propagation dynamics, multi-layer availability constraints, or AI agent failure behavior.

G) AI Agent Chaos Engineering

Emerging tools such as agent-chaos/balagan-agent inject real faults into AI agent environments to observe failure behavior. These approaches inject actual faults rather than simulating in-memory, do not model hallucination probability as a computable function of infrastructure state, and cannot exhaustively evaluate all failure combinations across infrastructure and agent layers.

2.3 Unmet Need

There exists no system that:

1. Models infrastructure topology entirely in memory without requiring access to real systems
2. Simulates thousands of failure scenarios automatically from a topology definition using formally-specified Labeled Transition System semantics with dependency-type-aware cascade rules
3. Computes mathematically rigorous availability limits across multiple independent constraint layers and constrains simulation results by physically realizable ceilings
4. Models dynamic behaviors including cascading failures with differential propagation based on dependency type, autoscaling responses, circuit breaker activation, and failover sequences
5. Simulates AI agent failure modes as functions of infrastructure state, including cross-layer cascade from infrastructure failure to agent hallucination to downstream decision errors, and generates infrastructure monitoring thresholds from hallucination probability computations
6. Produces quantitative resilience scores and infrastructure modification specifications that identify specific changes to relax binding constraints

The present invention addresses all of these needs.

3. SUMMARY OF THE INVENTION

The invention provides a computer-implemented system and method built on two core innovations:

Innovation 1: Formally-Specified Graph-Based Cascade Simulation with Multi-Layer Availability Constraints

An infrastructure topology is modeled as a directed graph stored entirely in computer memory, where nodes represent infrastructure components (including AI agents, LLM endpoints, tool services, and agent orchestrators) annotated with typed attributes including component type, replica count, MTBF, MTTR, and operational profile, and edges represent typed dependencies classified as required, optional, or asynchronous. Fault scenarios are automatically generated from the graph, and failure propagation is simulated using a cascade engine whose semantics are formally defined as a Labeled Transition System (LTS) comprising a state tuple of (health map, latency map, elapsed time, visited set) and typed transition rules that differentiate propagation behavior based on dependency type · required dependencies propagate failures unconditionally, optional dependencies degrade performance without cascade, and asynchronous dependencies propagate with configurable delay. Per-layer availability ceilings are computed across at least five independent constraint layers · hardware, software, theoretical, operational, and external SLA · and the binding constraint layer producing the minimum availability ceiling is identified. Simulation results exceeding any layer ceiling are flagged as physically unrealizable, identifying the binding constraint. The system generates infrastructure modification specifications identifying specific changes to relax the binding constraint layer. The method enables exhaustive combinatorial failure scenario analysis that is physically impossible with real-environment fault injection, and a visited-set mechanism guarantees simulation termination in $O(|C| + |E|)$ for acyclic dependency graphs.

Innovation 2: AI Agent Cross-Layer Failure Simulation

The system models AI agent failure as a function of infrastructure state through a formal hallucination probability model $H(a, D, I)$ that computes the probability of agent hallucination given the agent's base rate, data source dependencies with per-source weights, and the current infrastructure health state. The system simulates four-layer cross-layer cascade: (L1) infrastructure fault causes component failure; (L2) data source becomes unavailable or degraded; (L3) agent hallucination probability increases above threshold; (L4) hallucinated output propagates to downstream agents with compound probability amplification. The system computes compound hallucination probability for downstream agents incorporating amplification factors from upstream agent outputs, and generates infrastructure monitoring thresholds derived from hallucination probabilities, wherein a threshold for data source health degradation is computed such that agent hallucination probability remains below a configured acceptable level. The method reveals emergent failure modes invisible to infrastructure monitoring and AI evaluation benchmarks operating independently, by exposing causal chains from infrastructure faults through data availability

degradation to agent behavioral failures.

The system further provides multiple complementary simulation methodologies organized as a multi-engine architecture, including stochastic simulation (Monte Carlo), time-stepped dynamic simulation, agent-based modeling, discrete event simulation, Bayesian network analysis, Markov chain availability computation, fault tree analysis, and additional analytical engines.

4. DETAILED DESCRIPTION

4.1 System Architecture Overview

```
+-----------------------------------------------------+
|                    FaultRay System                   |
+-----------------------------------------------------+
|                                                      |
|  +-----------+    +----------------------------+     |
|  | Topology  |--->| In-Memory Directed Graph   |     |
|  | Definition|    | (networkx DiGraph)         |     |
|  | (YAML/    |    |                            |     |
|  |  Terraform/    | Nodes: Component instances |     |
|  |  Cloud API)    | Edges: Typed dependencies  |     |
|  +-----------+    +------------+---------------+     |
|                                 |                    |
|                   +-----------v----------+           |
|                   | Scenario Generator   |           |
|                   | (30 categories,      |           |
|                   |  2000+ scenarios)    |           |
|                   +-----------+----------+           |
|                                 |                    |
|         +----------------------+----------------+    |
|         v          v           v        v       v    |
|    +---------++---------++--------++-------++------+ |
|    |Cascade  ||Dynamic  || Ops    ||What-If||Capac.| |
|    |Engine   ||Engine   ||Engine  ||Engine ||Engine| |
|    |(LTS)    ||(time)   ||(days)  ||(param)||(sat.)| |
|    +----+----++----+----++---+----++---+---++--+---+ |
|         +----------+---------+-------+--------+      |
|                    v         v       v                |
|            +----------------------------+             |
|            | N-Layer Availability       |             |
|            | Constraint Model          |              |
|            | (5+ constraint layers)    |              |
|            +-------------+------------+               |
|                          v                            |
|            +----------------------------+             |
|            | AI Agent Cross-Layer       |             |
|            | Failure Simulation         |             |
|            | (H(a,D,I) + cascade)      |              |
|            +-------------+------------+               |
|                          v                            |
|            +----------------------------+             |
|            | Infrastructure Modification|             |
|            | Specification & Monitoring |             |
```

```
|            | Threshold Generation       |              |
|            +----------------------------+              |
|                                                       |
+-------------------------------------------------------+
```

4.2 Graph-Based Topology Model

The infrastructure topology is represented as a directed graph G = (V, E) where:

- **V (Vertices):** Each vertex represents an infrastructure component with the following typed attributes:
  - `type`: One of {load_balancer, web_server, app_server, database, cache, queue, storage, dns, external_api, ai_agent, llm_endpoint, tool_service, agent_orchestrator, custom}
  - `replicas`: Integer replica count for parallel redundancy modeling
  - `metrics`: Current resource utilization (CPU%, memory%, disk%, network connections, open files)
  - `capacity`: Resource limits (max connections, max RPS, connection pool size, timeout, retry multiplier)
  - `network`: Network characteristics (RTT, packet loss rate, jitter, DNS resolution time, TLS handshake time)
  - `runtime_jitter`: Application-level jitter sources (GC pause duration/frequency, scheduling jitter)
  - `operational_profile`: MTBF (hours), MTTR (minutes), deploy downtime (seconds)
  - `autoscaling`: HPA/KEDA configuration (min/max replicas, thresholds, delays, step size)
  - `failover`: Failover configuration (enabled, promotion time, health check interval, threshold)
  - `circuit_breaker`: Circuit breaker configuration (enabled, threshold, reset timeout)
  - `external_sla`: Provider SLA percentage for external dependencies
  - `team`: Operational team readiness (runbook coverage %, automation %)
- **E (Edges):** Each directed edge (u, v) represents that component u depends on component v, with attributes:
  - `dependency_type`: One of {requires, optional, async}
  - `weight`: Dependency strength (0.0 to 1.0)
  - `latency_ms`: Expected edge latency
  - `circuit_breaker`: Edge-level circuit breaker configuration
  - `retry_strategy`: Retry configuration (enabled, max retries, backoff strategy)
  - `singleflight`: Request coalescing configuration

The topology may be defined via a declarative YAML schema, import from Terraform state files, import from cloud provider APIs (AWS, GCP, Azure), import from Prometheus/monitoring metrics, import from Kubernetes cluster state, or an interactive step-by-step wizard.

4.3 Automated Fault Scenario Generation

The scenario generator produces fault scenarios across 30 categories, including but not limited to:

1. Single-component DOWN for each component in V
2. Traffic spike scenarios at multipliers {1.5x, 2x, 3x, 5x, 10x}

3. Pairwise combination failures: C(|V|, 2)
4. Triple combination failures: C(|V|, 3)
5. Component-type-specific faults (database replication lag, cache stampede, queue backpressure, DNS failure, external API timeout, etc.)
6. Network partition scenarios (zone isolation, cross-zone partition)
7. Gradual degradation scenarios (resource leak, configuration drift)
8. AI agent-specific scenarios (LLM endpoint rate limiting, tool service unavailability, context overflow under high-concurrency orchestration, coordinated grounding source failures)

4.4 Cascade Engine · Formal Specification

The Cascade Engine simulates failure propagation through the dependency graph. Its semantics are formally defined as a Labeled Transition System (LTS), providing provable correctness properties. The LTS formalism is not optional; it is the defining mechanism of the cascade engine, distinguishing this system from connectivity-based or heuristic failure models.

4.4.1 Labeled Transition System Definition

The Cascade Propagation Semantics (CPS) operates over a Labeled Transition System M = (S, S_0, Act, ->, F) where:

State S = (H, L, T, V) is a 4-tuple:

Symbol	Domain	Description
H	Component -> HealthStatus	Maps each component to its health status. HealthStatus = {HEALTHY, DEGRADED, OVERLOADED, DOWN}
L	Component -> R>=0	Maps each component to its accumulated latency in milliseconds
T	R>=0	Global elapsed time in seconds since fault injection
V	P(Component)	The set of already-visited components (monotonically growing)

Infrastructure Graph G = (C, E, dep, w, tau, cb):

Symbol	Domain	Description
C	Finite set	Set of infrastructure components (vertices)
E	C x C	Directed dependency edges. (a, b) in E means a depends on b
dep	E -> {requires, optional, async}	Dependency type function
w	E -> [0, 1]	Edge weight (criticality) function
tau	C -> R>=0	Timeout function: tau(c) is component c's timeout in milliseconds
cb	E -> {enabled, disabled}	Circuit breaker status for each edge

Initial State: Given a fault injection on component c_0 with fault type f:

```
S_0 = (H_0, L_0, 0, {c_0})
H_0(c) = { apply_direct_effect(c_0, f)   if c = c_0
         { HEALTHY                         otherwise
L_0(c) = { latency_direct(c_0, f)        if c = c_0
         { 0                               otherwise
```

Terminal States: A state S = (H, L, T, V) is terminal when: (1) queue exhaustion · no unvisited components are reachable from any failed component; (2) depth limit · propagation depth reaches D_max = 20; or (3) circuit breaker isolation · all remaining paths pass through tripped circuit breakers.

Actions: Act = {inject, propagate, timeout, trip_cb, degrade, terminate}

4.4.2 Transition Rules

The LTS defines eight transition rules that govern cascade behavior. Critically, these rules differentiate propagation based on dependency type · a distinction absent from connectivity-based approaches that treat all edges identically.

Rule 1: Fault Injection (Initial)

```
H(c_0) = HEALTHY
```

```
    ----------------------------------------------------------
    (H, L, 0, {}) --[inject(c_0, f)]--> (H', L', 0, {c_0})
    where H'(c) = { effect(f)  if c = c_0      L'(c) = { lat(f)  if
c = c_0
                  { H(c)       otherwise               { 0
otherwise
    effect: FaultType -> HealthStatus
    effect(COMPONENT_DOWN)             = DOWN
    effect(DISK_FULL)                  = DOWN
    effect(MEMORY_EXHAUSTION)          = DOWN
    effect(NETWORK_PARTITION)          = DOWN
    effect(CONNECTION_POOL_EXHAUSTION) = DOWN
    effect(CPU_SATURATION)             = OVERLOADED
    effect(TRAFFIC_SPIKE)              = OVERLOADED
    effect(LATENCY_SPIKE)              = DEGRADED
```

Rule 2: Cascade Propagation (Required Dependency, Single Replica)

```
    (c', c) in E    dep(c', c) = requires    c' not in V
    H(c) in {DOWN, OVERLOADED}    replicas(c') = 1    depth < D_max

-------------------------------------------------------------------
    (H, L, T, V) --[propagate(c, c', h')]--> (H[c' |-> h'], L[c'
|-> l'], T + dt, V u {c'})
    where h' = { DOWN        if H(c) = DOWN
               { OVERLOADED  if H(c) = OVERLOADED and
utilization(c') > 70%
               { DEGRADED    if H(c) = OVERLOADED and
utilization(c') <= 70%
          l' = { tau(c')                     if h' = DOWN
               { edge_latency(c',c) * 3.0    if h' = OVERLOADED
               { edge_latency(c',c) * 2.0    if h' = DEGRADED
```

Rule 3: Cascade Propagation (Required Dependency, Multiple Replicas)

```
    (c', c) in E    dep(c', c) = requires    c' not in V
    H(c) = DOWN    replicas(c') > 1    depth < D_max
    ---------------------------------------------------
    (H, L, T, V) --[degrade(c', "replicas absorb")]--> (H[c' |->
DEGRADED], L, T + 5, V u {c'})
```

When the dependent has multiple replicas, a DOWN dependency causes degradation (not failure), because remaining replicas absorb the load. Cascade does not continue propagating from c' in this case.

Rule 4: Optional Dependency

```
    (c', c) in E    dep(c', c) = optional    c' not in V
    H(c) = DOWN    depth < D_max
    -------------------------------------------------------
    (H, L, T, V) --[degrade(c', "optional dep down")]--> (H[c' |->
DEGRADED], L, T + 10, V u {c'})
```

Rule 5: Async Dependency

```
    (c', c) in E    dep(c', c) = async    c' not in V
    H(c) = DOWN    depth < D_max
```

```
    -------------------------------------------------------
    (H, L, T, V) --[degrade(c', "async queue buildup")]--> (H[c'
|-> DEGRADED], L, T + 60, V u {c'})
```

Rule 6: Circuit Breaker Trip

```
    (c', c) in E    cb(c', c) = enabled    c' not in V
    accumulated_latency(c) + edge_latency(c', c) > tau(c')
    ---------------------------------------------------------------
    (H, L, T, V) --[trip_cb(c, c')]--> (H[c' |-> DEGRADED], L[c'
|-> l_acc], T, V u {c'})
```

When a circuit breaker is enabled and the accumulated latency exceeds the dependent's timeout, the circuit breaker trips, the dependent is marked DEGRADED, and cascade stops through this path.

Rule 7: Timeout Propagation

```
    (c', c) in E    c' not in V
    cb(c', c) = disabled
    accumulated_latency(c) + edge_latency(c', c) > tau(c')
    ---------------------------------------------------------------
    (H, L, T, V) --[timeout(c', l_acc)]--> (H[c' |-> DOWN], L[c'
|-> l_acc], T, V u {c'})
```

Rule 8: Termination

```
    forall c in C \ V : (no applicable rule from Rules 2-7)
    -------------------------------------------------------
    (H, L, T, V) --[terminate]--> (H, L, T, V)
```

4.4.3 Correctness Properties

Theorem 1 (Termination): CPS terminates for all finite infrastructure graphs. Each non-terminal transition strictly decreases the measure mu(S) = |C \ V|. Since mu(S) >= 0 and mu(S) is a natural number, the sequence of non-terminal transitions is finite. At most |C| non-terminal transitions can occur.

Theorem 2 (Time Complexity): For acyclic graphs, CPS terminates in O(|C| + |E|) time. Each component is visited at most once, and for each visited component, all dependents are examined. For cyclic graphs with depth limit, termination is guaranteed in O(min(|C|, D_max) * |E|) steps.

Theorem 3 (Monotonicity of Failure): Once a component transitions to DOWN, it cannot return to HEALTHY, DEGRADED, or OVERLOADED within that simulation run. Health can only worsen or stay the same. Formally, for every transition S --[a]--> S' and every component c: H'(c) >= H(c) in the partial order HEALTHY < DEGRADED < OVERLOADED < DOWN. This ensures conservative worst-case analysis.

Theorem 4 (Causality): A component can transition from HEALTHY to a degraded state only if at least one of its dependencies has a non-HEALTHY status. Every failure in the simulation has an explainable causal chain back to the injected fault, with no spontaneous failures.

Theorem 5 (Circuit Breaker Correctness): If a circuit breaker is enabled on edge (c', c) and accumulated latency exceeds tau(c'), then: (1) cascade is stopped at c'; (2) c' is marked DEGRADED (not DOWN); (3) components downstream of c' are not affected by this cascade path.

Theorem 6 (Dependency Type Attenuation): The maximum cascade depth through optional or async dependency edges is 1 (they do not propagate further), because these rules produce DEGRADED status, which does not trigger recursive propagation.

Theorem 7 (Blast Radius Bound): For any fault injection, the number of affected components is at most min(|C|, reachable(c_0)) where reachable(c_0) is the set of components transitively reachable from c_0 via reverse

dependency edges.

4.4.4 Direct Effect Computation

For each fault f applied to component c, the direct effect is computed based on the fault type as defined in Rule 1. The likelihood score (0.2 to 1.0) is computed based on the current state proximity to the failure condition.

4.4.5 Latency Cascade Simulation

A specialized cascade mode simulates latency propagation through the dependency graph:

- Accumulated latency is tracked through each hop
- Circuit breakers are evaluated at each edge (cascade stopped if latency exceeds timeout)
- Connection pool exhaustion is computed when accumulated latency causes request pileup
- Retry storms are modeled as connection multiplication factors
- Singleflight (request coalescing) is modeled as a load reduction factor

4.4.6 Severity Scoring

Cascade severity is computed as:

```
severity = impact_score * spread_score * 10.0 * likelihood
```

Where:

- impact_score = (DOWN_count * 1.0 + OVERLOADED_count * 0.5 + DEGRADED_count * 0.25) / affected_count
- spread_score = affected_count / total_components
- Caps: no cascade (single component) -> max 3.0; <30% spread -> max 6.0; degraded-only -> max 4.0

4.4.7 Validation Against Real Incidents

CPS predictions have been validated against documented cascade failure patterns from real-world incidents. Given the topology and dependency configuration of each incident, CPS predicts the observed cascade path. Validation covers:

| Incident Pattern | CPS Mechanism | Formal Rule | |---|---|---|---| | Sequential cascade through single points of failure | _propagate with requires deps, replicas=1 | Rule 2 | | Degradation absorption via replicas | _propagate with replicas > 1 | Rule 3 | | Circuit breaker containment | trip_cb in latency cascade | Rule 6 | | Timeout-driven cascade | simulate_latency_cascade BFS | Rule 7 | | Traffic redistribution overload | simulate_traffic_spike | Rule 1 (OVERLOADED) | | Graceful degradation via optional deps | Optional/async dep attenuation | Rules 4-5 |

Backtest against 18 documented public cloud incidents (AWS us-east-1 2021, AWS S3 2017, Meta BGP 2021, Cloudflare 2022, GCP 2019, Azure 2023, and others) achieves: Avg Precision 1.000, Avg Recall 1.000, Avg F1 Score 1.000, Avg Severity Accuracy 0.819 (with shared infrastructure modeling).

4.5 N-Layer Availability Constraint Model · Formal Specification

The N-Layer Availability Constraint Model provides mathematically distinct availability ceilings that constrain cascade simulation results. The core insight is that system availability is bounded by multiple independent factors, each of which imposes a ceiling that cannot be exceeded regardless of improvements in other layers. When a cascade simulation predicts an availability exceeding any layer ceiling, the result is flagged as physically unrealizable, and the binding constraint layer · the layer producing the minimum ceiling · is identified as the target for infrastructure modification.

Mathematical Formulation:

For a system with components C = {c_1, c_2, ..., c_n} and dependency graph G = (C, E):

Layer 1 (Hardware Limit):

```
For each component c_i:
  A_single(c_i) = MTBF(c_i) / (MTBF(c_i) + MTTR(c_i))
  A_tier(c_i) = 1 - (1 - A_single(c_i))^replicas(c_i)
  If failover enabled:
    fo_events/year = (365.25 * 24 / MTBF(c_i)) * replicas(c_i)
    fo_downtime_fraction = fo_events * (promotion_time +
detection_time) / (365.25 * 24 * 3600)
    A_tier(c_i) = A_tier(c_i) * (1 - fo_downtime_fraction)
A_hw = product{c_i in critical_path(G)} A_tier(c_i)
```

Where critical_path(G) includes all components that have at least one requires-type dependent or are leaf nodes.

Layer 2 (Software Limit):

```
A_sw = min(1 - (deploy_freq * avg_deploy_downtime / period +
human_error_rate + config_drift_rate), A_hw)
```

Layer 3 (Theoretical Limit):

```
A_theoretical = A_hw * (1 - avg_packet_loss) * (1 -
avg_gc_fraction)
```

This is the irreducible physical noise floor from network packet loss rates, garbage collection pause fractions, and kernel scheduling jitter, applied as multiplicative penalties.

Layer 4 (Operational Limit):

```
effective_response = (mean_response / coverage) * runbook_factor *
automation_factor
A_ops = 1 - (incidents/year * effective_response / 8760)
```

Where effective response time is adjusted by on-call coverage percentage, runbook coverage, and automation level.

Layer 5 (External SLA Cascading):

```
A_external = product{c_i in external_deps} SLA(c_i)
```

This establishes the hard ceiling imposed by third-party service availability.

N-Layer Generalization:

The model is extensible to N layers. Additional layers include but are not limited to: Layer 6 (Geographic Limit) · availability ceiling imposed by geographic distance and data residency requirements; Layer 7 (Economic Limit) · availability ceiling imposed by budget constraints where cost increases exponentially per additional nine; Layer N (Custom Domain-Specific Limit) · any domain-specific constraint imposing an independent availability ceiling.

Effective System Availability:

```
A_system = min(A_layer1, A_layer2, ..., A_layerN)
```

Binding Constraint Identification:

```
binding_layer = argmin over k (A_layer_k)
```

The binding constraint layer is the layer producing the minimum availability ceiling. The system generates an infrastructure modification specification targeting the binding layer, identifying specific parameter changes (e.g., "

increase MTBF of component X from 720h to 2000h" or "add 2 replicas to component Y") required to relax the binding constraint and shift the bottleneck to the next most constraining layer.

Cascade Path Availability Computation:

The model integrates with the cascade engine's dependency graph to compute availability along each cascade path. For a cascade path P = [c_1, c_2, ..., c_k] where each c_i depends on c_{i-1}, the path availability is:

```
A_path(P) = product{i=1}^{k} A_tier(c_i) *
attenuation(dep_type(c_i, c_{i-1}))
```

Where attenuation is 1.0 for requires, 0.3 for optional, and 0.1 for async dependencies.

Simulation Result Constraining:

After the cascade engine computes simulation-predicted availability for each scenario, the N-Layer model constrains the results:

```
If A_simulation > A_system:
    Flag result as "physically unrealizable"
    Report: "Simulation predicts {A_simulation} but {binding_layer}
imposes ceiling of {A_system}"
    Generate modification specification for binding_layer
```

This integration ensures that optimistic simulation predictions are bounded by physical reality, preventing unrealizable availability claims.

4.6 AI Agent Cross-Layer Failure Simulation · Formal Specification

The system extends the infrastructure topology model to include AI-specific component types and failure modes, enabling resilience evaluation of AI agent architectures within the same in-memory simulation framework. This is the first system to model infrastructure components and AI agents in a single directed graph, enabling simulation of how infrastructure faults propagate through data availability layers to affect agent behavior, and generating infrastructure monitoring thresholds from the resulting hallucination probability computations.

4.6.1 AI Component Types

Four component types extend the graph model:

- **ai_agent**: An LLM-powered agent that processes requests using tools and makes autonomous decisions. Modeled as a vertex with attributes for context window size (tokens), supported tool list, fallback policy (degrade/fail/retry), maximum retry count, and base hallucination rate h_0(a).
- **llm_endpoint**: An LLM API endpoint (Anthropic, OpenAI, Google, self-hosted) with rate limit thresholds, SLA constraints, token-per-minute quotas, and cost-per-token attributes.
- **tool_service**: An external tool or API that agents invoke (database queries, web search, MCP servers), modeled with availability SLA, response time distribution, and payload size constraints.
- **agent_orchestrator**: A multi-agent coordination layer supporting sequential, parallel, and hierarchical orchestration patterns, modeled with coordination protocol type, timeout policy, and partial-failure handling strategy.

4.6.2 Hallucination Probability Model H(a, D, I)

Definition. Let H(a, D, I) denote the probability of hallucination for agent a, given:

- D = set of data sources available to agent a (grounding databases, retrieval indices, tool outputs)
- I = infrastructure state (mapping of each component to {HEALTHY, DEGRADED, DOWN, OVERLOADED})

Base Hallucination Rate. Every agent a has an intrinsic base hallucination rate h_0(a) in [0, 1], reflecting the model 's inherent tendency to produce ungrounded outputs even when all data sources are available.

Data Source Dependency Weights. For each data source d in D(a), w(d) in [0, 1] defines how critical data source d is to agent a's grounding.

Per-Source Hallucination Contribution. For each data source d:

```
If status(d, I) = HEALTHY:
    h_d = h_0(a)                                        (no
additional risk)
If status(d, I) = DOWN:
    h_d = h_0(a) + (1 - h_0(a)) * w(d)                  (full
dependency risk)
If status(d, I) = DEGRADED:
    h_d = h_0(a) + (1 - h_0(a)) * w(d) * delta          (partial
risk)
    where delta in (0, 1) is the degradation factor (default: 0.5)
If status(d, I) = OVERLOADED:
    h_d = h_0(a) + (1 - h_0(a)) * w(d) * omega
(overload risk)
    where omega in (0, 1) is the overload factor (default: 0.3)
```

This formula ensures: when h_0 = 0 and w(d) = 1, h_d = 1 (total hallucination when fully dependent source is down); when h_0 = 1, h_d = 1 regardless (already hallucinating); h_d is always in [h_0(a), 1].

Combined Hallucination Probability (independence assumption):

```
H(a, D, I) = 1 - product{d in D(a), status(d,I) != HEALTHY} (1 -
h_d)
```

When all data sources are HEALTHY: H(a, D, I) = h_0(a). When no data sources exist: H(a, D, I) = h_0(a).

Properties: (1) Monotonicity · H increases as more data sources fail; (2) Boundedness · H(a, D, I) in [h_0(a), 1]; (3) Compositionality · each data source contributes independently; (4) Infrastructure-dependent · H is a function of I, linking agent behavior to infrastructure state.

4.6.3 Cross-Layer Cascade Model

The system defines four layers of cross-layer cascade:

Layer	Name	Domain
L1	Infrastructure	Physical/virtual component failures
L2	Data Availability	Grounding source accessibility
L3	Agent Behavior	AI-specific failure modes
L4	Downstream Impact	Propagation to consumers

Formal Transition Model:

```
L1: Infrastructure fault F occurs on component c
    |
    For each data source d reachable from c in dependency graph G:
        status(d, I) transitions from HEALTHY to {DEGRADED, DOWN,
OVERLOADED}
L2: Data source d becomes unavailable/degraded
    |
    For each agent a where d in D(a):
        H(a, D, I) is recomputed using updated status(d, I)
        If H(a, D, I) > threshold_hallucination:
            agent a enters DEGRADED state (hallucination mode)
L3: Agent a produces unreliable output
```

```
    |
    For each downstream consumer b that receives output from a:
        If b is an agent:
            b's effective input quality degrades
            H(b, D', I) increases (tainted input acts as degraded
data source)
        If b is a system/user:
            Impact propagates as corrupted data
L4: Compound cascade
    |
    Agent-to-agent propagation follows the same graph traversal
    as infrastructure cascades, but in the agent dependency
subgraph
```

4.6.4 Agent Failure Taxonomy (10 Modes)

ID	Failure Mode	Health Impact	Trigger
hallucination	Confident but incorrect output not grounded in data	DEGRADED	Data source dependency severed; H(a,D,I) > threshold
context_overflow	Input exceeds token limit	DOWN	Cumulative tokens > max_context_tokens
token_exhaustion	API budget depleted	DOWN	consumed_tokens >= budget_tokens
prompt_injection	Adversarial input hijacks behavior	DEGRADED	Unsanitized external input with directive text
agent_loop	Infinite tool call cycles without progress	DOWN	tool_calls > max_iterations without progress
confidence_miscalibration	High confidence on incorrect outputs	DEGRADED	Distribution shift; degraded grounding
cot_collapse	Reasoning degrades mid-generation	DEGRADED	Token budget pressure; context saturation
output_amplification	Hallucination consumed by downstream agent	DEGRADED to DOWN	Agent-to-agent flow without validation gates
grounding_staleness	Data sources available but outdated	DEGRADED	Cache TTL expiry; replication lag
llm_rate_limit	API throttling causes timeout cascade	OVERLOADED	Traffic spike; concurrent invocations

Each failure mode is formally defined with conditions for triggering, health impact on the agent vertex, and recovery characteristics.

4.6.5 Agent-to-Agent Cascade Propagation

When agents form a directed acyclic graph (DAG) of data flow, a failure in one agent propagates through the graph:

```
Given agent graph G_agent = (A, E) where:
    A = set of agents
    E = set of directed edges (a_i -> a_j) meaning a_j consumes
output of a_i
For a source agent a_s with hallucination probability H(a_s):
    For each edge (a_s -> a_t) in E:
        amplification_factor(a_s, a_t) = 1.0 if a_t has no
independent verification
                                        = v(a_t) in [0, 1) if a_t
can partially verify
        H_effective(a_t) = 1 - (1 - H(a_t, D, I)) * (1 - H(a_s) *
amplification_factor(a_s, a_t))
```

Compound Cascade Probability for a chain of agents [a_1, a_2, ..., a_n]:

```
H_chain(a_n) = 1 - product{i=1}^{n} (1 - H_effective(a_i))
```

Properties: (1) Amplification · H_chain(a_n) >= max(H(a_i)) for all i; (2) Monotonic growth · adding agents never decreases compound risk; (3) Mitigation · validation gates between agents reduce amplification_factor, bounding cascade growth.

4.6.6 Infrastructure Monitoring Threshold Generation

The system derives infrastructure monitoring thresholds from the hallucination probability model. For each AI agent a with a configured maximum acceptable hallucination probability H_max(a), the system computes the minimum data source health required to maintain H(a, D, I) <= H_max(a):

```
For each data source d in D(a):
    threshold_health(d, a) = minimum health state of d such that
        H(a, D, I) <= H_max(a) given current state of all other
data sources
    monitoring_alert(d, a) = {
        trigger: status(d, I) degradation approaching
threshold_health(d, a),
        severity: proportional to w(d) * (H_max(a) - h_0(a)),
        action: "Data source {d} health degradation threatens agent
{a} hallucination threshold"
    }
```

These monitoring thresholds bridge the gap between infrastructure monitoring (which tracks component health) and AI agent reliability (which depends on grounding data availability), providing concrete, actionable alerts that infrastructure operators can use to prevent agent behavioral degradation before it occurs.

4.6.7 Grounding Data Dependency Tracking

The system maintains a dependency map from each ai_agent vertex to its grounding data sources (tool_service vertices providing database queries, retrieval indices, web search results). When the cascade engine processes a fault that affects any tool_service vertex, the system:

1. Identifies all ai_agent vertices that depend on the affected tool_service
2. Recomputes H(a, D, I) for each affected agent using the updated infrastructure state
3. If H(a, D, I) exceeds the configured hallucination threshold, transitions the agent to DEGRADED state
4. Propagates the degradation through the agent dependency subgraph using the compound cascade probability model

4.6.8 Tool Call Loop Detection

The system detects tool call loops by monitoring simulated request patterns on ai_agent vertices. A loop is detected when: (1) the agent's simulated request count exceeds max_iterations within a time window; (2) no state progression is detected (the agent is not making progress toward task completion); (3) the same tool is being invoked repeatedly with similar parameters. Upon detection, the agent vertex transitions to DOWN state and cascade propagation continues to its dependents.

4.7 Additional Simulation Engines (Summary)

The system provides over one hundred complementary simulation and analysis engines beyond the core cascade, availability, and AI agent engines. The following table summarizes key engines by category:

| Category | Engine | Key Technique | |----------|--------|---------------| | **Core Simulation** | Dynamic Engine | Time-stepped simulation with traffic patterns, autoscaling, failover, circuit breakers | | | Operations Engine | Multi-day stochastic simulation with MTBF/MTTR Poisson events | | | What-If Engine | Parametric sensitivity analysis across system parameters | | | Capacity Engine | Resource saturation prediction with quorum evaluation | | **Probabilistic** | Monte Carlo Simulation | Stochastic availability estimation via repeated random sampling | | | Bayesian Network Engine | Conditional failure probability via Bayes' theorem | | | Markov Chain Engine | Steady-state availability via 3-state CTMC | | | Common Cause Failure Analyzer | Beta-factor correlated failure probability | | **Discrete/Agent-Based** | Discrete Event Simulation | Priority-queue event processing with exact

temporal resolution | | | Agent-Based Model (ABM) | Autonomous agents with probabilistic cascade, emergent behavior | | | Cellular Automata | Threshold-based local rules with pattern classification | | | System Dynamics | Continuous ODE-based health with Euler integration | | | Petri Net Engine | Place/Transition net with reachability and deadlock detection | | **Formal Methods** | Fault Tree Analysis (FTA) | Top-down AND/OR/VOTING gate analysis, minimal cut sets | | | Reliability Block Diagram | Series/parallel availability composition | | | Event Tree Analysis (ETA) | Inductive forward risk assessment from initiating events | | | Model Checker | Exhaustive state-space CTL verification (AG, EF, AF) | | | Causal Inference Engine | Structural Causal Model, do-calculus, counterfactual reasoning | | **ML/AI Prediction** | RNN/LSTM Predictor | Temporal dependency capture from simulation-synthesized data | | | Random Forest Predictor | Bagged tree ensembles for non-linear failure prediction | | | Anomaly Autoencoder | Unsupervised anomaly detection via reconstruction error | | | Transformer Predictor | Self-attention with interpretable attention weights | | | GNN Cascade Predictor | MPNN architecture trained on CascadeEngine results | | | ARIMA Predictor | Classical time-series forecasting | | | AdaBoost Predictor | Sequential decision stump boosting | | **Optimization** | Genetic Algorithm | Population-based evolutionary worst-case discovery | | | Simulated Annealing | Metropolis criterion single-solution search | | | Particle Swarm Optimization | Swarm-based cooperative search | | | Pareto Optimizer (NSGA-II) | Multi-objective Pareto front computation | | | Bayesian Optimizer | GP surrogate with Expected Improvement | | **Generative** | GAN Scenario Generator | Adversarial training for novel failure patterns | | | VAE Scenario Generator | ELBO-optimized latent space interpolation | | | Chaos Fuzzer | AFL-inspired scenario mutation and discovery | | | RL Scenario Generator | Q-learning agent for scenario discovery | | **Specialized** | Failure Pattern Clustering | K-means++ with elbow-method cluster selection | | | Queueing Theory Engine | M/M/1, M/M/c, Erlang-C, Little's Law | | | Extreme Value Theory | GEV distribution, return levels, tail risk | | | Game Theory Analyzer | Nash equilibrium, minimax defense strategies | | | Fuzzy Logic Engine | Mamdani inference with centroid defuzzification | | | Survival Analysis | Kaplan-Meier, Weibull, remaining useful life | | **Security** | Security Resilience Engine | Attack simulation with defense matrix | | | Compound Security-Failure Simulator | Combined infrastructure + security scenarios | | | Attack Surface Analyzer | Exposure surface computation | | | Supply Chain Engine | Dependency supply chain risk | | **Financial/Compliance** | Financial Risk Engine | VaR95, expected annual loss, mitigation ROI | | | Compliance Framework Simulation | SOC2, ISO27001, PCI-DSS, DORA, HIPAA, GDPR | | | FMEA Engine | Failure Mode and Effects Analysis with RPN | | | SLA Mathematical Provability | Formal SLA achievability proof | | **Operational** | Blast Radius Predictor | Pre-simulation impact estimation with confidence intervals | | | Chaos Experiment Recommender | Prioritized real-world experiment recommendations | | | Backtest Engine | Historical incident validation with auto-calibration | | | Digital Twin Synchronization | Continuous shadow simulation from live metrics | | | Incident Response Simulator | Response workflow simulation | | | Disaster Recovery Orchestrator | DR procedure simulation | | **Infrastructure** | Circuit Breaker Tuner | Optimal circuit breaker configuration | | | Timeout Budget Analyzer | End-to-end timeout budget computation | | | Connection Pool Analyzer | Pool sizing and exhaustion prediction | | | Load Shedding Engine | Graceful degradation planning | | | Multi-Tenant Isolation Verifier | Noisy-neighbor and data isolation verification | | | DNS Resilience Analyzer | DNS infrastructure failure analysis | | | Service Mesh Resilience Engine | Service mesh failure mode analysis | | **Integration Pipelines** | Multi-Engine Consensus | Cross-engine voting and confidence computation | | | Compliance Audit Pipeline | Simulate -> comply -> audit workflow | | | Cascade-Cost-Remediation | Simulate -> cost -> remediate workflow | | | Backtest-Calibration Loop | Iterative prediction validation and parameter tuning | | | Full Lifecycle Automation | Discover -> simulate -> report -> remediate -> validate | | | Inverse Optimizer | Target SLA -> required changes computation |

Each engine operates on the same in-memory directed graph representation described in Section 4.2, enabling unified analysis across all methodologies. The multi-engine consensus pipeline (Section 4.7, Integration Pipelines) combines results from multiple engines through voting, computing agreement scores with divergence detection.

4.8 Resilience Score Computation

The system aggregates simulation results into a quantitative resilience score (0-100) enabling comparison across different infrastructure designs. The score integrates cascade severity distribution, SPOF count, availability layer analysis, and redundancy coverage.

5. ALTERNATIVE EMBODIMENTS AND EXTENSIONS

5.1 Machine Learning-Enhanced Scenario Generation

In an alternative embodiment, the automated scenario generator is augmented with a machine learning model (including but not limited to large language models, graph neural networks, or reinforcement learning agents) that analyzes historical incident data, identifies failure combinations that are statistically likely but not covered by rule-based generation, learns from simulation results to prioritize high-impact scenarios, and generates novel failure scenarios.

5.2 Digital Twin Continuous Synchronization

In an alternative embodiment, the in-memory topology model is continuously synchronized with a real infrastructure environment through real-time metric ingestion, automatic topology updates when changes are detected, continuous simulation re-evaluation, and drift detection. A further refinement feeds the metric stream to a dependency inference engine that discovers emergent relationships not present in the original topology declaration.

5.3 Natural Language Infrastructure Definition

In an alternative embodiment, the infrastructure topology is defined through natural language input, where a language model interprets descriptions and automatically generates the corresponding graph topology.

5.4 Simulation-Driven Infrastructure Remediation

In an alternative embodiment, the system automatically generates Infrastructure-as-Code modifications from simulation results, ranked by cost-effectiveness, and applies approved remediations through pull request generation.

5.5 Multi-Cloud Correlated Failure Analysis

In an alternative embodiment, the system models infrastructure spanning multiple cloud providers and analyzes correlated failures across provider boundaries.

5.6 Cost-Constrained Resilience Optimization

In an alternative embodiment, the system performs multi-objective optimization to find optimal configurations under cost constraints, including Pareto frontier computation and budget allocation optimization.

6. CLAIMS

Independent Claims

Claim 1. A computer-implemented method for evaluating infrastructure resilience, comprising:

- (a) constructing, in computer memory, a directed graph representation of an infrastructure topology, wherein nodes represent infrastructure components annotated with component attributes including at least component type, replica count, MTBF, MTTR, and operational profile, and edges represent typed dependency relationships between components, each edge classified as one of required, optional, or asynchronous;
- (b) simulating failure propagation entirely in computer memory without affecting any real infrastructure, using a formally-defined labeled transition system comprising a state tuple of (health map, latency map, elapsed time, visited set) and typed transition rules that differentiate propagation behavior based on dependency type, wherein:
    - required dependencies propagate failures unconditionally from a failed component to its single-replica dependents,
    - optional dependencies degrade performance of dependent components without further cascade propagation,

    - asynchronous dependencies propagate degradation with configurable delay reflecting queue buildup behavior;
- (c) computing per-layer availability ceilings across at least five independent constraint layers · a hardware layer computing single-instance availability from MTBF and MTTR with parallel redundancy, a software layer accounting for deployment downtime and human error rate, a theoretical layer computing the irreducible physical noise floor from packet loss and garbage collection pauses, an operational layer modeling human factor availability from incident frequency and response time, and an external SLA layer computing a hard ceiling from third-party service availability · and determining the binding constraint layer as the layer producing the minimum availability ceiling;
- (d) constraining the simulation results of step (b) by said per-layer ceilings of step (c), wherein a simulation-predicted availability exceeding any layer ceiling triggers an indication that the result is physically unrealizable and identifies the binding constraint layer;
- (e) generating an infrastructure modification specification identifying specific changes to component attributes in the directed graph that would relax the binding constraint layer, thereby enabling higher achievable system availability;
- wherein the method enables exhaustive combinatorial failure scenario analysis that is physically impossible with real-environment fault injection due to the exponential growth of failure combinations, and wherein a visited-set mechanism in the labeled transition system guarantees simulation termination in O(|C| + |E|) time for acyclic dependency graphs where |C| is the number of components and |E| is the number of dependency edges.

Claim 2. A computer-implemented method for simulating AI agent failure behavior as a function of infrastructure state, comprising:

- (a) modeling AI agent components within an infrastructure dependency graph stored in computer memory, each AI agent vertex having a base hallucination rate attribute and weighted dependency edges to data source vertices, wherein each data source vertex represents a grounding information source with an associated dependency weight indicating the criticality of that source to the agent's output quality;
- (b) simulating infrastructure faults affecting data source vertices within said dependency graph and computing health states for each affected vertex, wherein each vertex is assigned a health status from the set {HEALTHY, DEGRADED, OVERLOADED, DOWN};
- (c) computing a hallucination probability H(a, D, I) for each AI agent a as a function of the agent's base hallucination rate h_0(a), data source dependency weights w(d), and infrastructure health states of data sources, using the formula: for each unavailable data source d with health status DOWN, computing per-source hallucination contribution h_d = h_0(a) + (1 - h_0(a)) * w(d), and computing the combined hallucination probability as H(a, D, I) = 1 - product over all non-healthy data sources d of (1 - h_d);
- (d) simulating cross-layer cascade propagation across four layers: (L1) infrastructure fault causes component failure, (L2) data source becomes unavailable causing grounding data loss for dependent agents, (L3) agent hallucination probability exceeds a configured threshold causing the agent to enter a degraded behavioral state, (L4) degraded agent output propagates to downstream agent consumers through the agent dependency subgraph;
- (e) computing compound hallucination probability for each downstream agent a_t that consumes output from an upstream agent a_s, incorporating an amplification factor reflecting whether the downstream agent has independent verification capability, using the formula: H_effective(a_t) = 1 - (1 - H(a_t, D, I)) * (1 - H(a_s) * amplification_factor(a_s, a_t));
- (f) generating infrastructure monitoring thresholds derived from said hallucination probabilities, wherein for each AI agent and each of its data source dependencies, a threshold for data source health degradation is computed such that the agent's hallucination probability remains below a configured acceptable level, said thresholds providing actionable alerts to infrastructure operators;
- wherein said method reveals emergent failure modes invisible to infrastructure monitoring and AI evaluation benchmarks operating independently, by exposing causal chains from infrastructure

faults through data availability degradation to agent behavioral failures that propagate through agent-to-agent dependency chains.

Dependent Claims

Claim 3. The method of Claim 1, wherein the labeled transition system comprises eight transition rules: (1) fault injection establishing initial failure state, (2) cascade propagation through required dependencies with single replica causing dependent transition to DOWN, (3) cascade propagation through required dependencies with multiple replicas causing dependent transition to DEGRADED without further propagation due to remaining replicas absorbing load, (4) optional dependency degradation without further cascade, (5) asynchronous dependency degradation with delayed time delta reflecting queue buildup, (6) circuit breaker trip when accumulated latency exceeds dependent timeout causing cascade termination through that path, (7) timeout propagation when circuit breaker is disabled causing dependent transition to DOWN, and (8) termination when no further applicable rules exist.

Claim 4. The method of Claim 1, wherein simulating failure propagation further comprises modeling circuit breaker containment, wherein for each edge with an enabled circuit breaker configuration, when accumulated latency through the dependency chain exceeds the dependent component's timeout threshold, the circuit breaker trips, the dependent component transitions to DEGRADED rather than DOWN, and cascade propagation is halted through that path, thereby bounding the blast radius of latency-induced failures.

Claim 5. The method of Claim 1, wherein simulating failure propagation further comprises replica-aware degradation calculation, wherein for each dependent component with a replica count greater than one and a required dependency to a DOWN component, the dependent transitions to DEGRADED rather than DOWN, modeling the capacity of remaining replicas to absorb load from the failed dependency, and cascade propagation does not continue through the degraded multi-replica component.

Claim 6. The method of Claim 1, wherein computing per-layer availability ceilings comprises: computing the hardware layer ceiling as A_hw = product of A_tier(c_i) for all components in the critical path, where A_single(c_i) = MTBF(c_i) / (MTBF(c_i) + MTTR(c_i)) and A_tier(c_i) = 1 - (1 - A_single(c_i))^replicas(c_i), with failover penalty applied as a multiplicative factor; computing the software layer ceiling as A_sw = min(1 - (deploy_ frequency * avg_deploy_downtime / period + human_error_rate + config_drift_rate), A_hw); computing the theoretical layer ceiling as A_theoretical = A_hw * (1 - avg_packet_loss) * (1 - avg_gc_fraction); computing the operational layer ceiling as A_ops = 1 - (incidents_per_year * effective_response / 8760); and computing the external layer ceiling as A_external = product of SLA(c_i) for all external dependencies.

Claim 7. The method of Claim 2, wherein the AI agent components are associated with a failure taxonomy comprising at least ten failure modes: hallucination, context overflow, token exhaustion, prompt injection, tool call loop, confidence miscalibration, chain-of-thought collapse, output amplification, grounding data staleness, and rate limit cascade, each mode having a defined health impact on the agent vertex, a trigger condition based on infrastructure state, and a recovery characteristic.

Claim 8. The method of Claim 2, further comprising computing a compound cascade probability for a chain of agents [a_1, a_2, ..., a_n] as H_chain(a_n) = 1 - product over i=1 to n of (1 - H_effective(a_i)), wherein H_chain is monotonically non-decreasing as agents are added to the chain, demonstrating that longer agent chains amplify compound failure risk.

Claim 9. The method of Claim 1, further comprising simulating latency cascade propagation through the dependency graph by tracking accumulated latency through each dependency hop, evaluating circuit breaker thresholds at each edge, computing connection pool exhaustion when accumulated latency causes request pileup, modeling retry storms as connection multiplication factors, and modeling singleflight request coalescing as a load reduction factor.

Claim 10. The method of Claim 1, further comprising simulating autoscaling response to failure conditions by modeling horizontal pod autoscaler (HPA) and event-driven autoscaler (KEDA) configurations, including minimum and maximum replica counts, scaling thresholds, scale-up and scale-down delays, and step size, and computing the time-dependent replica count during failure propagation to determine whether autoscaling can mitigate cascade

effects before they propagate to dependent components.

Claim 11. The method of Claim 1, further comprising performing what-if parameter sweep analysis by systematically varying component attributes including replica counts, MTBF values, timeout thresholds, and circuit breaker configurations across a defined parameter space, executing the failure simulation of step (b) and the availability computation of step (c) for each parameter combination, and identifying the parameter changes that produce the largest improvement in resilience score per unit of infrastructure cost.

Claim 12. The method of Claim 1, embodied as a system comprising: at least one processor; a memory coupled to the at least one processor; and instructions stored in the memory that, when executed by the at least one processor, cause the system to perform the method of Claim 1.

APPENDIX A: Prior Art Differentiation

A.1 Detailed Prior Art Comparison

Krasnovsky & Zorkin (Connectivity-Based Resilience Model): Krasnovsky and Zorkin model infrastructure resilience using graph connectivity metrics · vertex connectivity and edge connectivity · to measure how many components or links must fail before a system becomes disconnected. Their model treats all dependencies as equivalent graph edges, computes static connectivity metrics, and does not simulate dynamic failure cascade behavior. The present invention differs fundamentally: (i) the cascade engine uses a formally-defined Labeled Transition System with eight typed transition rules, not connectivity metrics; (ii) dependency types (required, optional, asynchronous) produce qualitatively different propagation behaviors · required dependencies cascade unconditionally while optional and asynchronous dependencies attenuate; (iii) circuit breaker containment modeling halts cascade through protected paths; (iv) the LTS state tuple tracks health maps, latency maps, elapsed time, and visited sets, enabling dynamic simulation of cascade propagation through time; (v) the system integrates multi-layer availability constraints and AI agent failure modeling, neither of which is addressed by connectivity-based approaches.

GMOR (2016) (Generalized Multi-Hazard Outage Risk): GMOR models physical infrastructure (power grids, water systems) exposure to natural hazards using fragility curves and Monte Carlo simulation. It analyzes physical damage propagation through infrastructure networks but does not model IT infrastructure dependency types, does not distinguish between required/optional/asynchronous software dependencies, does not incorporate circuit breaker containment or timeout-based cascade, and has no AI agent behavior modeling. The present invention operates in the IT infrastructure domain with software-specific dependency semantics.

Dell US11356324B2 (Graph Analysis for Fault Injection Target Selection): Dell's patent describes a system that analyzes infrastructure topology graphs to select optimal targets for real-environment fault injection. The graph analysis is used for target prioritization, not for in-memory failure simulation. After targets are selected, faults are injected into actual infrastructure. The present invention performs complete failure simulation entirely in memory without affecting real infrastructure, simulates cascade propagation dynamics using formal LTS semantics, computes multi-layer availability ceilings, and models AI agent failure behavior · none of which is addressed by Dell's target selection system.

agent-chaos/balagan-agent (AI Agent Chaos Engineering): This tool injects real faults into AI agent environments to observe agent failure behavior. It operates by actually disrupting agent infrastructure · modifying API endpoints, injecting latency, restricting access to tools. The present invention does not inject real faults; it models hallucination probability H(a, D, I) as a computable function of infrastructure state entirely in memory, enabling exhaustive evaluation of failure combinations without risking real agent operation. The hallucination probability model · connecting infrastructure health states to agent behavioral degradation through formal mathematical functions · is not present in any real-fault-injection approach.

Isograph Reliability Workbench (Single-Layer Availability): Isograph and similar RAMS tools compute availability using hardware reliability models (MTBF/MTTR with redundancy). They operate within a single constraint layer. The present invention computes independent availability ceilings across five or more constraint

layers (hardware, software, theoretical, operational, external SLA), identifies the binding constraint layer, and generates infrastructure modification specifications targeting that specific layer · a multi-layer constraint analysis absent from single-layer reliability tools.

Bell-LaPadula Model: The Bell-LaPadula model is a formal state machine model for enforcing mandatory access control in computer security. It addresses information flow control (no read up, no write down), not infrastructure resilience or failure simulation. It is not relevant prior art for the present invention.

A.2 Novel Contributions Summary

| Aspect | Present Invention | Nearest Prior Art | Distinction | |--------|---------|---------|---------| | Cascade Engine | LTS with 8 typed transition rules, dependency-type differentiation | Krasnovsky (connectivity metrics) | Formal semantics vs. static metrics; dependency types vs. uniform edges | | Availability Model | 5+ independent constraint layers with binding constraint identification | Isograph (single-layer MTBF/MTTR) | Multi-layer ceiling analysis vs. single-layer computation | | AI Agent Failure | H(a,D,I) as computable function of infrastructure state | balagan-agent (real fault injection) | In-memory mathematical model vs. real-environment observation | | Simulation Scope | Exhaustive combinatorial in-memory | Dell US11356324B2 (graph for target selection) | Complete in-memory simulation vs. target selection for real injection | | Output | Infrastructure modification specifications + monitoring thresholds | All prior art | Actionable remediation targeting binding constraints |

APPENDIX B: Implementation Reference

The described system is implemented as the FaultRay software, available at https://github.com/mattyopon/faultray under the Business Source License 1.1 (BSL-1.1). The implementation was first committed on March 9, 2026. The implementation is in Python 3.11+ and utilizes the networkx library for graph operations and pydantic for data model validation. All patent rights are reserved by the inventor; the source license does not grant any patent rights for commercial or production use.

Key implementation files:

- Graph model: `src/faultray/model/graph.py`
- Component model: `src/faultray/model/components.py`
- Cascade Engine: `src/faultray/simulator/cascade.py`
- Availability Model: `src/faultray/simulator/availability_model.py`
- Scenario Generator: `src/faultray/simulator/scenarios.py`
- Dynamic Engine: `src/faultray/simulator/dynamic_engine.py`
- Monte Carlo Simulation: `src/faultray/simulator/monte_carlo.py`
- Security Resilience Engine: `src/faultray/simulator/security_engine.py`
- Financial Risk Engine: `src/faultray/simulator/financial_risk.py`
- Integration Pipelines: `src/faultray/simulator/integration_pipelines.py`
- 100+ additional engine files in `src/faultray/simulator/`

END OF PATENT APPLICATION